pgTap
Home: https://pgtap.org/documentation.html

*** Installation (Docker)**
Reference: https://qiita.com/RYA234/items/5644d55634f234017378

Step 1) Setup docker

Dockerfile

```
FROM postgres:17.5-bullseye

ENV UNAME=user
ENV GID=1000
ENV UID=1000

RUN groupadd -g $GID -o $UNAME
RUN useradd -m -u $UID -g $GID -G sudo -o -s /bin/bash $UNAME
RUN echo "$UNAME ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers



# pgtap
RUN apt-get update \
    && apt-get install -y build-essential git-core libv8-dev curl
postgresql-server-dev-$PG_MAJOR \
    && rm -rf /var/lib/apt/lists/*

# install pg_prove
RUN curl -LO http://xrl.us/cpanm \
    && chmod +x cpanm \
    && ./cpanm TAP::Parser::SourceHandler::pgTAP

# install pgtap
RUN git clone https://github.com/theory/pgtap.git \
    && cd /pgtap \
    && make \
    && make install
```

docker-compose.yml

```
version: "3.7"

services:
  playground-postgres:
    container_name: playground-postgres
    build:
      context: ./
      dockerfile: ./Dockerfile
    image: playground-postgres:latest
    environment:
      - POSTGRES_DB=playground
      - POSTGRES_USER=yuinishizawa
```

```
      - POSTGRES_HOST_AUTH_METHOD=trust                    # allow
login without password
      - POSTGRES_PORT=5432
    ports:
      - "54321:5432"
    volumes:
      - ./initdb.d:/docker-entrypoint-initdb.d
```

./initdb.d/0_init.sql

```
-- Step: user
CREATE ROLE tester
SUPERUSER
LOGIN;

-- Step: install
CREATE SCHEMA tap;
SET search_path=tap;
CREATE EXTENSION pgtap WITH SCHEMA tap;

-- Step: user settings
ALTER ROLE tester IN DATABASE playground
  SET search_path=tap,public;
GRANT ALL ON DATABASE playground TO tester;
```

Step 2: start docker
e.g.

```
# Step: start docker
$ docker compose -f docker-compose.yml up

# Step: PSQL into DB
$ psql -h localhost -p 54321 -U tester playground
```

Step 3: install extension and use

```
-- Check that tap is installed
> \dx
                 List of installed extensions
  Name   | Version |   Schema   |         Description
---------+---------+------------+------------------------------
 pgtap   | 1.3.4   | tap        | Unit testing for PostgreSQL
 plpgsql | 1.0     | pg_catalog | PL/pgSQL procedural language
(2 rows)
```

Step 3: run test

Write test

```
-- test.sql
BEGIN;
SELECT plan(1);

SELECT pass('Hello');
SELECT fail('Hello');

SELECT * FROM finish(TRUE);
ROLLBACK;

-- NOTE: exit code
-- Need to throw explicitly for exit code
--
-- Reference: https://github.com/theory/pgtap/issues/198
DO $$ BEGIN ASSERT num_failed() = 0; END $$;
```

Run

```
$ psql -h localhost -p 54322 -U tester \
  --no-psqlrc \
  --no-align \
  --pset pager=off \
  --pset tuples_only=true \
  --set ON_ERROR_STOP=1 \
  -f ./test.sql \
  playground

BEGIN
1..1
ok 1 - Hello
not ok 2 - Hello
# Failed test 2: "Hello"
# Looks like you planned 1 test but ran 2
ROLLBACK
```

*** API**

Basic

```
SELECT plan(10);                                    -- 10=# of expected
tests
SELECT finish();
SELECT finish(TRUE);                                -- throw exception
if TRUE
```

Scalar

```
SELECT pass( :description );
SELECT pass( );
```

```
SELECT fail( :description );
SELECT fail( );

SELECT ok( :boolean, :description );
SELECT ok( :boolean );

SELECT
is(   :have, :want, :description );                    -- :have
= :want
SELECT is(   :have, :want );
SELECT isnt( :have, :want, :description );
SELECT isnt( :have, :want );

SELECT
matches( :have, :regex, :description );                -- :have
~ :regex
SELECT matches( :have, :regex );
SELECT imatches( :have, :regex, :description );            -- case
insensitive
SELECT imatches( :have, :regex );
SELECT doesnt_match(  :have, :regex, :description );
SELECT doesnt_match(  :have, :regex );
SELECT doesnt_imatch( :have, :regex, :description );
SELECT doesnt_imatch( :have, :regex );

SELECT
alike(  :this, :like, :description );              -- :this
LIKE :like
SELECT alike(  :this, :like );
SELECT ialike( :this, :like, :description );               -- case
insensitive
SELECT ialike( :this, :like );
SELECT unalike(  :this, :like, :description );
SELECT unalike(  :this, :like );
SELECT unialike( :this, :like, :description );
SELECT unialike( :this, :like );

SELECT cmp_ok( :have, :op, :want, :description );          -- e.g.
cmp_ok(1, '<', 2, '1<2')
SELECT cmp_ok( :have, :op, :want );

SELECT isa_ok( :have, :regtype, :name );                   -- e.g.
isa_ok(1, 'integer' '1 is an int')
SELECT isa_ok( :have, :regtype );
```

```
-- Table
SELECT results_eq( :sql,    :sql,    :description );
SELECT results_eq( :sql,    :sql                  );
SELECT results_eq( :sql,    :array,  :description );
SELECT results_eq( :sql,    :array                );
```

```
SELECT results_eq( :cursor, :cursor, :description );
SELECT results_eq( :cursor, :cursor              );
SELECT results_eq( :sql,    :cursor, :description );
SELECT results_eq( :sql,    :cursor              );
SELECT results_eq( :cursor, :sql,    :description );
SELECT results_eq( :cursor, :sql                  );
SELECT results_eq( :cursor, :array,  :description );
SELECT results_eq( :cursor, :array                );

SELECT results_ne( :sql,    :sql,    :description );
SELECT results_ne( :sql,    :sql                  );
SELECT results_ne( :sql,    :array,  :description );
SELECT results_ne( :sql,    :array                );
SELECT results_ne( :cursor, :cursor, :description );
SELECT results_ne( :cursor, :cursor               );
SELECT results_ne( :sql,    :cursor, :description );
SELECT results_ne( :sql,    :cursor               );
SELECT results_ne( :cursor, :sql,    :description );
SELECT results_ne( :cursor, :sql                  );
SELECT results_ne( :cursor, :array,  :description );
SELECT results_ne( :cursor, :array                );



-- Set: no duplicates
SELECT set_eq( :sql, :sql,   :description );
SELECT set_eq( :sql, :sql                 );
SELECT set_eq( :sql, :array, :description );
SELECT set_eq( :sql, :array               );
SELECT set_ne( :sql, :sql,   :description );
SELECT set_ne( :sql, :sql                 );
SELECT set_ne( :sql, :array, :description );
SELECT set_ne( :sql, :array               );
ELECT set_has( :sql, :sql, :description );
SELECT set_has( :sql, :sql );
SELECT set_hasnt( :sql, :sql, :description );
SELECT set_hasnt( :sql, :sql );

-- Bag: allow duplicates
SELECT bag_eq( :sql, :sql,   :description );
SELECT bag_eq( :sql, :sql                 );
SELECT bag_eq( :sql, :array, :description );
SELECT bag_eq( :sql, :array               );
SELECT bag_ne( :sql, :sql,   :description );
SELECT bag_ne( :sql, :sql                 );
SELECT bag_ne( :sql, :array, :description );
SELECT bag_ne( :sql, :array               );
SELECT bag_has( :sql, :sql, :description );
SELECT bag_has( :sql, :sql );
SELECT bag_hasnt( :sql, :sql, :description );
SELECT bag_hasnt( :sql, :sql );

SELECT is_empty( :sql, :description );
SELECT is_empty( :sql );
```

```
SELECT isnt_empty( :sql, :description );
SELECT isnt_empty( :sql );

SELECT row_eq( :sql, :record, :description );
SELECT row_eq( :sql, :record );
```

```
--
CREATE TABLE t (
  id INT,
  message VARCHAR(10)
);

-- case: compare as columns
results_eq(
$$ SELECT * FROM t $$,

$$ VALUES (1, 'a')$$,                  -- GOTCHA: type mismatch
$$ VALUES (1, 'a'::VARCHAR(10))$$,     -- OK
$$ VALUES ((ROW(1, 'a')::t).*)$$,      -- OK. Cast to t first.

'test t'
)

-- case: compare as ROW aka composite object
results_eq(
$$ SELECT t FROM t $$,
$$ VALUES (ROW(1, 'a')::t)$$,      -- OK
'test t'
)
```

   ==========

```
This
is
code
Monaco 14
```